INTERNSHIP TECHNICAL DOCUMENTATION

VitalGuard

An IoT-Based Vital Signs, Fall Detection &
GPS Geofencing Monitoring System

Firmware Design & Code Analysis Report

Prepared By

M. Nishanth *23881A04G9*
O. Venkat Sharma *23881A04H7*
N. Pooja *23881A04H6*

Platform: ESP32
Sensors: MAX30105, MPU6050, GPS (NEO-series)
Display: SH1106 OLED
Date: July 2026

Contents

1 Abstract

VitalGuard is a wearable, ESP32-based health and safety monitor intended for field personnel such as soldiers, lone workers, or patients requiring remote supervision. The firmware integrates four subsystems on a single microcontroller: photoplethysmography-based heart-rate and blood-oxygen (SpO2) estimation via a MAX30105 sensor, fall detection via an MPU6050 accelerometer, GPS-based geofencing using a custom point-in-polygon routine, and a live Wi-Fi dashboard served directly from the device, alongside a local OLED read-out.

The firmware is compact, functional, and demonstrates a good grasp of multi-sensor integration on a resource-constrained platform. It successfully combines real-time signal sampling, a geometric containment algorithm, and a self-hosted web server into a single Arduino sketch. At the same time, the implementation has a number of correctness, robustness, and security gaps that are typical of an early-stage prototype and worth addressing before field deployment.

2 Introduction

Remote monitoring of vital signs and physical safety is a growing requirement across defence, industrial, and healthcare settings. Personnel operating alone or in hazardous environments benefit from a wearable device that can continuously track heart rate, blood oxygen saturation, sudden falls, and geographic location, and relay this information to a supervisor without requiring dedicated infrastructure.

VitalGuard was developed under the domain of Embedded Systems and the Internet of Things (IoT). The firmware combines an ESP32 microcontroller with a MAX30105 pulse-oximetry sensor, an MPU6050 accelerometer, a serial GPS module, and an SH1106 OLED display to create a self-contained wearable that estimates BPM and SpO2, detects falls, evaluates whether the wearer is inside or outside a predefined geofence, and exposes all of this data through a locally hosted Wi-Fi dashboard as well as an on-device display.

3 System Overview

At a high level, the device continuously performs four loops of work inside a single Arduino `loop()` function, without the use of an RTOS or task scheduler:

- **Wi-Fi Dashboard:** Serves an HTTP dashboard over Wi-Fi (root page and a `/data` JSON endpoint), polled by the browser every second.
- **GPS Parsing & Geofencing:** Parses incoming NMEA sentences from a serial GPS module and, when a fix is valid, tests the current location against a fixed 10-point geofence polygon.
- **Vitals Sampling:** Samples the MAX30105 IR/Red channels to estimate heart rate (BPM) and blood oxygen saturation (SpO2).
- **Fall Detection:** Reads MPU6050 acceleration and compares its magnitude against a fixed threshold to latch a fall alarm.
- **Local Display:** Refreshes a 128×64 OLED with BPM, SpO2, and zone status roughly every 10 ms.

Architecturally, this is a single-threaded, polling-based design: every subsystem is serviced once per `loop()` iteration. This is simple to reason about, but it means that any slow operation – for example, the geofence check's trigonometric calculations – delays every other subsystem, including the web server's responsiveness and the sensor sampling rate.

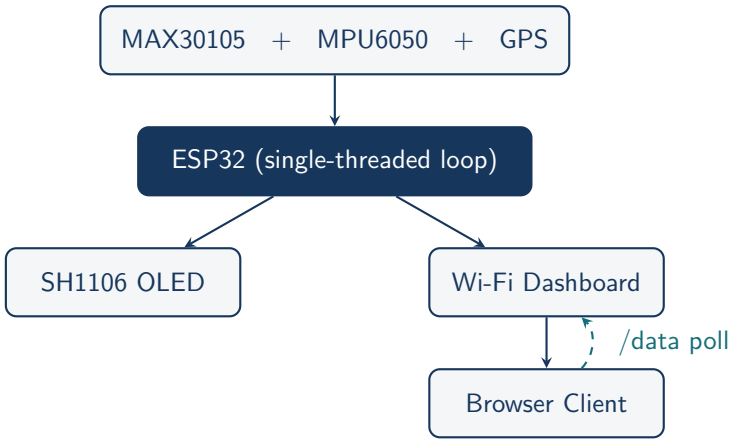

Figure 1: System Architecture Diagram of VitalGuard

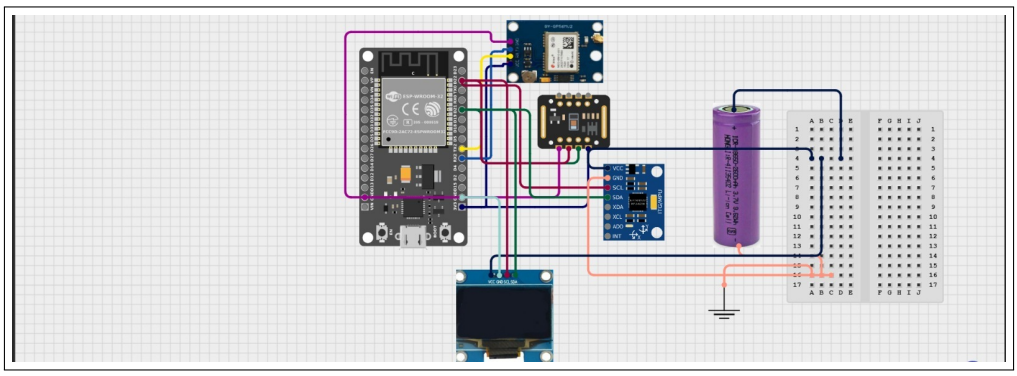

Figure 2: VitalGuard Circuit Wiring Diagram (ESP32, GPS, MAX30105, MPU6050, OLED, Li-ion battery)

4 Hardware & Library Stack

The sketch integrates five distinct peripherals through well-established Arduino libraries.

Component	Library Used	Role in System
ESP32	WiFi.h, WebServer.h	Main controller; hosts the Wi-Fi access point client and web server
MAX30105	MAX30105.h	Optical pulse-oximetry sensor for IR/Red light used to derive BPM and SpO2
MPU6050	MPU6050.h	6-axis accelerometer/gyroscope used for fall detection
GPS module	TinyGPS++.h	Provides latitude/longitude for geofence evaluation
SH1106 OLED	Adafruit_GFX.h, Adafruit_SH110X.h	128×64 local display of BPM, SpO2, and zone status

Table 1: Hardware and Library Stack Used

A serial-based GPS module is wired on ESP32 hardware serial UART2 (RX=16, TX=17), and I^2C peripherals (OLED, MAX30105, MPU6050) share bus pins 21/22, which is a sensible and

conventional pin allocation for this class of board.

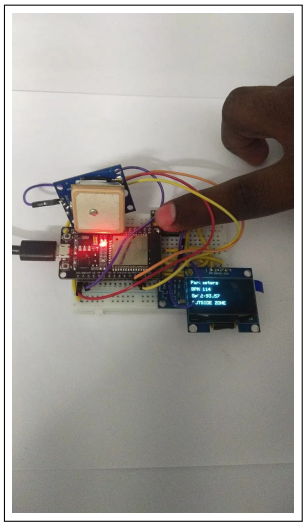

Figure 3: Assembled prototype – ESP32, GPS module, and OLED on breadboard

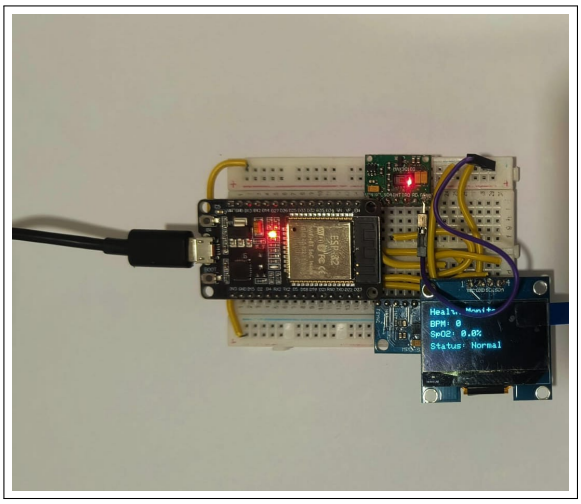

Figure 4: Close-up showing MAX30105 sensor and OLED read-out

5 Module-Level Analysis

5.1 Wi-Fi Dashboard & Web Server

The device hosts a self-contained HTML/CSS/JS dashboard as an in-memory string, served at `"/"`, while a second route, `"/data"`, returns a hand-built JSON string polled by the browser every second via `fetch()`. This is a lightweight and effective pattern for embedded dashboards and avoids pulling in a heavier JSON library.

- **Strength:** single-file deployment – no SPIFFS/LittleFS filesystem or external assets required.
- **Strength:** the polling interval (1 s) is a reasonable balance between responsiveness and ESP32 load.
- **Gap:** Wi-Fi credentials (SSID, password) are hard-coded in plaintext in source, and the `setup()` Wi-Fi connection loop has no timeout or offline fallback – if credentials are wrong or the network is unreachable, the device blocks indefinitely on startup and never begins monitoring.
- **Gap:** the `/data` endpoint has no authentication, so any device on the same network can read live health and location data.

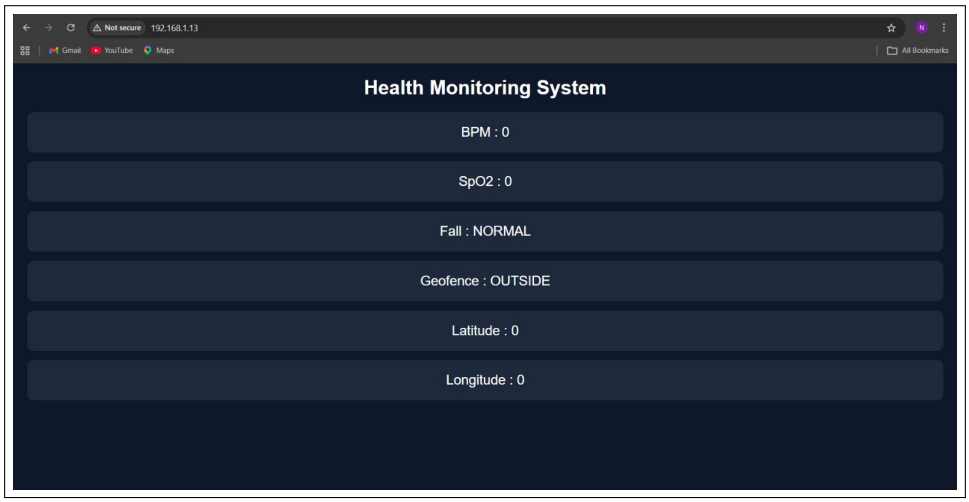

Figure 5: VitalGuard web dashboard in the idle state (before sensor data arrives)

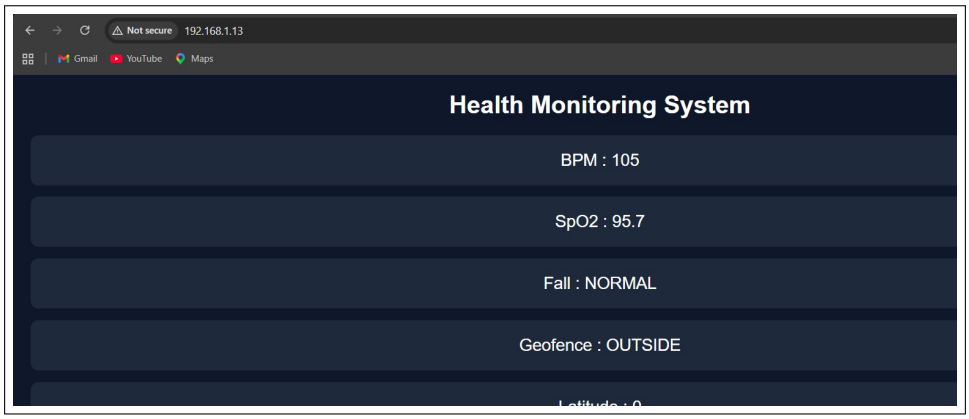

Figure 6: VitalGuard web dashboard showing live BPM and SpO2 readings

5.2 Heart Rate & SpO2 Estimation

BPM is derived from a simple peak-detection scheme: the change in IR reading between consecutive samples (`diff`) is compared to a fixed threshold (`PEAK_THRESHOLD = 300`), and the time between accepted peaks is converted to instantaneous BPM, which is then smoothed with a 5-sample rolling buffer. SpO2 is estimated using the classic ratio-of-ratios approach: exponential moving averages track the DC component of the Red and IR channels, the AC component is derived by subtraction, and the ratio R feeds a linear formula, $\text{SpO2} = 110 - 25R$.

- **Strength:** the rolling-average BPM buffer and exponential DC filtering are recognizable, reasonable signal-processing choices for a microcontroller with no floating-point DSP hardware.
- **Gap:** peak detection on a raw first-difference signal with a single fixed threshold is sensitive to motion artifact and ambient light noise; a proper implementation would add a band-pass or moving-average filter stage before peak detection.
- **Gap:** the SpO2 formula is a widely used illustrative approximation, not a calibrated clinical curve – the coefficients 110 and 25 are typically obtained per-device via calibration against a reference oximeter.
- **Correctness concern:** SpO2 is clamped to the range [90, 100]. In a health/safety monitor this is risky, because hypoxia (clinically significant desaturation) occurs below 90%

– clamping the floor at 90 means the one reading range that matters most for triggering an alert can never be displayed or reported.

5.3 Fall Detection

Fall detection compares the magnitude of the 3-axis acceleration vector (normalized to g) against a fixed threshold, `FALL_THRESHOLD = 2` (i.e., $2g$). Once triggered, `fallLatched` is set permanently – there is no code path that ever clears it.

- **Strength:** using acceleration magnitude rather than a single axis correctly makes the check orientation-independent, which is appropriate for a wearable that can be mounted in different positions.
- **Gap:** a single-threshold, single-sample trigger is prone to false positives from ordinary vigorous movement (running, jumping, dropping the arm quickly) and false negatives for slow slumps/collapses that don't produce a sharp acceleration spike. Production fall-detection algorithms typically combine a free-fall phase (near-zero g) with a subsequent impact spike and a post-impact stillness check.
- **Gap:** because the flag is never reset, the system has no way to acknowledge and clear an alarm short of a full device reboot – worth adding a physical reset button or a remote 'acknowledge' command from the dashboard.

5.4 GPS Geofencing (Point-in-Polygon)

The `pip()` function implements an angle-summation point-in-polygon test: for each edge of a fixed 10-vertex polygon, it computes the angle subtended at the current GPS position and sums these angles across all edges. If the total approaches 360°, the point is classified as inside the fence; the code checks for a tolerance band of 355°–365° to allow for floating point error.

- **Strength:** the angle-summation method is a legitimate and well-known point-in-polygon technique, and using a tolerance band around 360° instead of exact equality is the correct way to handle floating-point accumulation error.
- **Gap – performance:** each call to `pip()` performs 10 square roots and 10 arccosine calls. On an ESP32 running without hardware trigonometric acceleration, this is comparatively expensive to run on every single `loop()` iteration (potentially hundreds of times per second), and will measurably slow down sensor sampling and web server responsiveness. Since GPS updates typically arrive at only 1 Hz, this check only needs to run once per new fix.
- **Gap – geometric approximation:** latitude/longitude degrees are used directly as if they were Cartesian coordinates. Over the small area implied by the sample polygon this introduces negligible error, but the approach would degrade for larger geofences or fences far from the equator, where a degree of longitude corresponds to a different physical distance than a degree of latitude.
- **Gap – status flicker:** there is no hysteresis on the boundary. A GPS position jittering by a few meters near the fence edge can cause the reported zone status to flicker between INSIDE and OUTSIDE in successive updates.

5.5 OLED Display & Main Loop Timing

The OLED is refreshed every iteration of `loop()`, immediately after a fixed `delay(10)`. Sensor initialization calls (`particleSensor.begin()`, `mpu.initialize()`, `display.begin()`) do not check their return values, so if any peripheral is missing, miswired, or fails to initialize, the sketch will continue running silently with dead sensors rather than reporting a fault.

- **Gap:** `delay(10)` combined with unbounded-cost operations elsewhere (geofence check) means the loop period is not deterministic, which matters for consistent BPM peak-timing accuracy.
- **Gap:** no visible error/fault state on the OLED or the dashboard if a sensor fails to initialize – a user would only notice a stuck 'BPM: 0' after the fact, as seen in Figure 5.

6 Results and Discussion

The assembled prototype (Figures 3 and 4) was tested end-to-end: the ESP32 successfully joined the target Wi-Fi network, sampled the MAX30105 and MPU6050 sensors, parsed NMEA sentences from the GPS module, and served a live dashboard. In the idle state (Figure 5), the dashboard correctly reports zeroed BPM/SpO2 and a default `Fall: NORMAL` / `Geofence: OUTSIDE` status before a finger is placed on the sensor and before a GPS fix is acquired. Once a finger was placed on the MAX30105, the system produced live readings of BPM = 105 and SpO2 = 95.7 (Figure 6), consistent with the on-device OLED read-out shown in Figure 3, which displayed BPM 114 and SpO2 93.57 alongside the geofence status during a separate test run.

These results confirm that the core multi-sensor pipeline – optical vitals sensing, inertial fall detection, GPS parsing, and Wi-Fi dashboarding – functions correctly end-to-end on the ESP32. The remaining opportunities for improvement, catalogued below, concern robustness and safety-oriented refinement rather than the fundamental architecture.

7 Key Findings

The table below consolidates the most significant observations from the module-level review, ordered roughly by risk to the system's core purpose (accurate, timely health/safety alerting).

Area	Observation	Recommendation
Security	Wi-Fi SSID/password hard-coded; `/data` endpoint unauthenticated	Move credentials to a separate untracked config header or NVS; add a shared token / basic auth to the web routes
Reliability	No timeout on Wi-Fi connect loop in `setup()`	Add a retry/timeout with fallback to an offline mode (e.g., SoftAP config portal) so the device is usable without the target network
Reliability	Sensor init return values ignored	Check `begin()`/`initialize()` results and surface a fault state on the OLED and dashboard
Health data accuracy	SpO2 clamped to [90, 100]; formula uncalibrated	Remove the artificial floor so true low readings are visible; calibrate the R-to-SpO2 curve against a reference pulse oximeter
Signal processing	BPM peak detection on unfiltered first difference	Add a moving-average or band-pass filter ahead of peak detection to reduce motion-artifact false triggers

Area	Observation	Recommendation
Fall detection	Single-threshold, single-sample trigger; alarm never resets	Combine free-fall + impact + post-impact stillness checks; add an explicit acknowledge/reset path
Performance	`pip()` runs every loop iteration (10× sqrt + 10× acos)	Only re-run the geofence check when a new GPS fix arrives (GPS updates at ∼1 Hz)
Geofence UX	No hysteresis at the fence boundary	Require N consecutive consistent readings, or add a small buffer margin, before flipping status
Code hygiene	Unused variable `sat`; `#define M_PI` redefines an existing `math.h` constant	Remove dead code; rely on the standard `math.h` `M_PI` instead of redefining it
Architecture	Single monolithic `loop()`, no scheduler	Consider a `millis()`-based cooperative scheduler (or FreeRTOS tasks, which ESP32 already runs) to decouple subsystem timing

Table 2: Key Findings and Recommendations

8 Strengths Worth Highlighting

- Successfully integrates five heterogeneous peripherals (I^2C sensors, UART GPS, Wi-Fi, display) into a single, working Arduino sketch – a non-trivial integration exercise.
- Uses recognizable, textbook-correct signal processing building blocks: exponential moving average filtering for DC removal, ratio-of-ratios for SpO2, and rolling-average smoothing for BPM.
- The geofence check uses a legitimate computational-geometry technique (angle-summation point-in-polygon) with an appropriately tolerant threshold band for floating-point comparison, rather than a naive/incorrect approach.
- The self-hosted dashboard is a clean, dependency-light way to visualize live data without needing a companion mobile app.
- Orientation-independent fall detection via acceleration magnitude, rather than a single axis, is the right design instinct.

9 Suggested Next Steps

For a student project or internship deliverable, the current sketch is a strong proof of concept. To move it toward a more robust or deployable version, the following incremental roadmap is suggested:

- **Short term:** fix the SpO2 clamp, remove the unused variable and redundant `M_PI` define, add sensor-init error handling, and gate `pip()` behind "new GPS fix received".
- **Medium term:** replace hard-coded Wi-Fi credentials with a config file or captive-portal setup flow; add basic authentication to the web dashboard.

- **Medium term:** add a simple digital filter (e.g., moving average) ahead of BPM peak detection, and add hysteresis/debounce to the geofence status.
- **Longer term:** refactor the fall-detection logic into a small state machine (free-fall → impact → stillness) and add a manual acknowledge/reset mechanism; consider logging fall/geofence events to onboard flash or the cloud for post-incident review.
- **Longer term:** split the monolithic `loop()` into cooperative, `millis()`-timed tasks (or ESP32 FreeRTOS tasks) so that the web server, GPS parsing, sensor sampling, and display refresh each run on their own independent cadence.

10 Conclusion

VitalGuard demonstrates solid foundational embedded systems skills: multi-sensor integration, basic digital signal processing, computational geometry for geofencing, and a self-hosted web interface, all running on a single ESP32. The core logic for each subsystem is conceptually sound, and the assembled prototype (Figures 3–6) confirms that the full sensing-to-dashboard pipeline works end-to-end. The main opportunities for improvement lie in robustness (timeouts, error handling, filtering) and safety-oriented details (the SpO2 floor, fall-alarm reset, credential handling) rather than in the fundamental approach – all of which are natural and expected next steps for a prototype moving toward a field-ready device.